HW7. sys_call_table[] is in arch/x86/kernel/syscall_table_32.S. How many system calls does Linux 2.6 support? What are the system call numbers for exit, fork, execve, wait4, read, write, and mkdir? Find system call numbers for sys_ni_syscall, which is defined at kernel/sys_ni.c. What is the role of sys_ni_syscall?

> 우선 Linux 2.6은 0번부터 326까지 총 327개의 system call을 지원한다.
>
> 이는 `arch/x86/kernel/syscall_table_32.S`를 직접 확인해 보면 알 수 있다.

```
ENTRY(sys_call_table)
        .long sys_restart_syscall       /* 0 - old "setup()" system call, used for restarting */
        .long sys_exit
        .long sys_fork
        .long sys_read
        .long sys_write
        .long sys_open          /* 5 */
```

...

```
        .long sys_signalfd
        .long sys_timerfd_create
        .long sys_eventfd
        .long sys_fallocate
        .long sys_timerfd_settime       /* 325 */
        .long sys_timerfd_gettime
:set nu                                                         328,2-9       Bot
```

> 위의 파일을 직접 확인해보면 다음의 정보 또한 확인할 수 있다.

function	system call	system call number
exit	sys_exit	1
fork	sys_fork	2
execve	sys_execve	11
wait4	sys_wait4	114
read	sys_read	3
write	sys_write	4
mkdir	sys_mkdir	39

> 이때, `sys_ni_syscall`이라는 system_call이 여러개 눈에 띄는데 해당 시스템 콜은 `kernel/sys_ni.c`에 정의되어 있다.

```
asmlinkage long sys_ni_syscall(void)
{
        return -ENOSYS;
}
```

> 확인해 보면 `-ENOSYS`를 바로 return 하고있는 것을 볼 수 있는데, 이것은 아직 구현되지 않은 함수를 사용할 때 반환되는 오류코드이다.

HW8. Change the kernel such that it prints "length 17 string found" for each printf(s) when the length of s is 17. Run a program that contains a printf() statement to see the effect.

Idea

`printf(s)`는 결국 `write(1, s, strlen(s))`을 호출하고, 또 이 `write()`는 컴파일시 `syscall(4)`를 호출한다.

즉, `printf`함수를 수정하기 위해서는 syscall_table의 4번 함수인 `sys_write()`함수를 고쳐주어야 한다.

이 `sys_write()`함수는 `fs/read_write.c`에 존재한다.

Code

1. `fs/read_write.c/sys_write()`를 다음과 같이 고쳐준다.

```
asmlinkage ssize_t sys_write(unsigned int fd, const char __user * buf, size_t count)
{
        if (count == 17){
                printk("length 17 string found\n"); // check!!
        }
        struct file *file;
        ssize_t ret = -EBADF;
        int fput_needed;

        file = fget_light(fd, &fput_needed);
        if (file) {
                loff_t pos = file_pos_read(file);
                ret = vfs_write(file, buf, count, &pos);
                file_pos_write(file, pos);
                fput_light(file, fput_needed);
        }

        return ret;
}
```

2. 수정한 내용이 반영되도록 Recompile and Reboot한다.

3. 길이가 17인 문자열을 출력하는 코드를 만든다.

```
#include <stdio.h>

int main(){
        printf("length 17 string\n");
        return 0;
}
```

Result

결과를 확인하기 전 printk로 출력하는 내용을 볼 수 있도록 log level을 바꿔주어야 한다.

```
localhost HW # ./printf_test
length 17 string
localhost HW # echo 8 > /proc/sys/kernel/printk
localhost HW # ./printf_test
length 17 string found
length 17 string
```

그 후 코드를 확인해 본 결과 위와 같이 잘 동작하는 것을 알 수 있다.

HW9 You can call a system call indirectly with "syscall()". Write a program that prints "hello" in the screen using syscall.

```
write(1, "hi", 2);
syscall(4, 1, "hi", 2); // 4 is the system call number for "write" system call
```

Code

```
#include <stdio.h>
#include <string.h>
int main(void){
        const char* s = "hello\n";
        syscall(4, 1, s, strlen(s));
        return 0;
}
```

먼저 위와 같이 C의 `syscall()`함수를 이용하여 hello를 출력하는 프로그램을 짜 주었다.

```
localhost HW # gcc syscall_test.c -o syscall_test
localhost HW # ls
syscall_test  syscall_test.c
```

그리고 gcc를 이용하여 syscall_test실행파일을 만들어 주었다.

Result

```
localhost HW # ./syscall_test
hello
```

HW10. Create a new system call, my_sys_call with system call number 17 (system call number 17 is one that is not being used currently). Define my_sys_call() just before sys_write() in read_write.c Write a program that uses this system call. When the above program runs, the kernel should display "hello from my_sys_call".

Define My Syscall Number

Custom Syscall을 정의하기 위해서는 다음과 같은 과정을 거쳐야 한다.

1. `arch/x86/kernel/syscall_table_32.S`의 Sys_call_table에서 알맞은 17번 Index에 내 syscall의 이름을 정의해 준다.

gentoo - VMware Workstation 16 Player (Non-commercial use only)

```
ENTRY(sys_call_table)
        .long sys_restart_syscall       /* 0 - old "setup()" system call, used for restarting */
        .long sys_exit
        .long sys_fork
        .long sys_read
        .long sys_write
        .long sys_open          /* 5 */
        .long sys_close
        .long sys_waitpid
        .long sys_creat
        .long sys_link
        .long sys_unlink        /* 10 */
        .long sys_execve
        .long sys_chdir
        .long sys_time
        .long sys_mknod
        .long sys_chmod         /* 15 */
        .long sys_lchown16
        .long my_sys_call       /* old break syscall holder */
        .long sys_stat
        .long sys_lseek         17번 Sys_call_table
        .long sys_getpid        /* 20 */
        .long sys_mount
```

2. `fs/read_write.c`에 다음과 같이 `my_sys_call()`함수를 정의해 준다.

```
asmlinkage void my_sys_call(){
        printk("hello from my_sys_call\n");
}

asmlinkage ssize_t sys_write(unsigned int fd, const char __user * buf, size
```

3. Recompile and Reboot

(참고)
`asmlinkage`는 어셈블리 코드에서 이 함수를 직접 호출할 수 있다는 의미이다.

Result

`my_syscall_test`라는 c파일을 만들어 실행 결과를 확인해 보자.

```
#include <stdio.h>

void main(){
        syscall(17);
}
```

결과를 확인하기 전 printk로 출력하는 내용을 볼 수 있도록 log level을 바꿔주어야 한다.

```
localhost ~ # ./HW/my_syscall_test
localhost ~ # echo 8 > /proc/sys/kernel/printk
localhost ~ # ./HW/my_syscall_test
hello from my_sys_call
```

17번 syscall을 잘 사용할 수 있는 것을 확인할 수 있다.

HW10-1. Create another system call that will add two numbers given by the user.

Define My Syscall Number

1. `arch/x86/kernel/syscall_table_32.S`의 Sys_call_table에서 알맞은 32번 Index에 내 syscall의 이름을 정의해 준다.

```
gentoo - VMware Workstation 16 Player (Non-commercial use only)
Player
.long sys_mknod
.long sys_chmod          /* 15 */
.long sys_lchown16
.long my_sys_call        /* old break syscall holder */
.long sys_stat
.long sys_lseek
.long sys_getpid         /* 20 */
.long sys_mount
.long sys_oldumount
.long sys_setuid16
.long sys_getuid16
.long sys_stime          /* 25 */
.long sys_ptrace
.long sys_alarm
.long sys_fstat
.long sys_pause
.long sys_utime          /* 30 */
.long my_syscall_add     /* old stty syscall holder */
.long sys_ni_syscall
.long sys_access
.long sys_nice
.long sys_ni_syscall     /* 35 - old ftime syscall holder */
.long sys_sync
.long sys_kill
```

2. `fs/read_write.c`에 다음과 같이 `my_syscall_add()`함수를 정의해 준다.

```
asmlinkage int my_syscall_add(int a, int b){
        return a + b;
}

asmlinkage void my_sys_call(){
        printk("hello from my_sys_call\n");
}
```

3. Recompile and Reboot

```
localhost linux-2.6.25.10 # make bzImage
  CHK     include/linux/version.hh
  CHK     include/linux/utsrelease.h
  CALL    scripts/checksyscalls.sh
  CHK     include/linux/compile.h
Kernel: arch/x86/boot/bzImage is ready  (#20)
localhost linux-2.6.25.10 # cp arch/x86/boot/bzImage /boot/bzImage2
cp: overwrite '/boot/bzImage2'? yes
localhost linux-2.6.25.10 # reboot
```

Result

`my_syscall_test`라는 c파일을 만들어 실행 결과를 확인해 보자.

```
#include <stdio.h>

void main(){
        int a = 10;
        int b = 4;
        int sum = syscall(31, a, b);
        printf("sum is %d\n", sum);
}

```

> 아래의 그림과 같이 31번 syscall이 잘 동작하고 있는것을 확인할 수 있다.
>
> ```
> localhost HW # gcc my_syscall_add_test.c -o my_syscall_add_test
> my_syscall_add_test.c: In function 'main':
> my_syscall_add_test.c:3: warning: return type of 'main' is not 'int'
> localhost HW # ./my_syscall_add_test
> sum is 14
> ```

HW11. Modify the kernel such that it displays the system call number for all system calls. Run a simple program that displays "hello" in the screen and find out what system calls have been called. Also explain for each system call why that system call has been used.

> **Idea**
>
> system call이 호출될 때, 우선 ISR1이 호출된 후 ISR2가 호출된다.
>
> ISR1은 `arch/x86/kernel/entry_32.S`에 존재한다.
> 이때, `system_call`의 경우, 해당 코드에서 `index`를 사용해 `syscall_table`에 저장된 ISR2를 호출한다.
>
> 따라서, `system_call`을 호출할 때, 그 번호까지 같이 출력되도록 하기 위해서는
> ISR1이 호출되는 `arch/x86/kernel/entry_32.S`에서 `systemcall`의 index를 출력하는 함수를 추가해 주면 될 것이다.
>
> ---
>
> **Code**
>
> 1. `arch/x86/kernel/syscall_table_32.S`에 주어진 system call number를 출력하는 함수명 정의
>
> ```
> .long sys_chmod          /* 15 */
> .long sys_lchown16
> .long my_syscall         /* old break syscall holder */
> .long sys_stat
> .long sys_lseek
> .long sys_getpid         /* 20 */
> .long sys_mount
> .long sys_oldumount
> .long sys_setuid16
> .long sys_getuid16
> .long sys_stime          /* 25 */
> .long sys_ptrace
> .long sys_alarm
> .long sys_fstat
> .long sys_pause
> .long sys_utime          /* 30 */
> .long my_syscall_add     /* old stty syscall holder */
> .long my_syscall_num     /* old gtty syscall holder */
> .long sys_access
> .long sys_nice
> .long sys_ni_syscall     /* 35 - old ftime syscall holder */
> .long sys_sync
> ```
>
> 2. `fs/read_write.c`에 해당 함수 구현
>
> ```
> asmlinkage void my_syscall_num(int num){
>  printk("syscall[%d] is called!!\n", num);
> }
>
> asmlinkage int my_syscall_add(int a, int b){
>  return a + b;
> }
>
> asmlinkage void my_syscall(void){
>  printk("hello from my_syscall\n");
> }
> ```

3. `arch/x86/kernel/entry_32.S` 수정

```
ENTRY(system_call)
        RINGO_INT_FRAME                         # can't unwind into user space anyway
        pushl %eax                              # save orig_eax
        CFI_ADJUST_CFA_OFFSET 4
        SAVE_ALL
        GET_THREAD_INFO(%ebp)
                                                # system call tracing in operation / emulation
        /* Note, _TIF_SECCOMP is bit number 8, and so it needs testw and not testb */
        testw $(_TIF_SYSCALL_EMU|_TIF_SYSCALL_TRACE|_TIF_SECCOMP|_TIF_SYSCALL_AUDIT),TI_flags(%ebp)
        jnz syscall_trace_entry
        cmpl $(nr_syscalls), %eax
        jae syscall_badsys
syscall_call:
        /* call my_syscall_num */
        pushl %eax
        call my_syscall_num
        popl %eax
        /* ------------------- */

        call *sys_call_table(,%eax,4)
```

1. System_call이 호출되었을 때

3. call number를 레지스터에 저장
call number를 print
사용한 레지스터를 원상복구

2. sys_call_table을 조회하기 전에

4. Recompile and Reboot

```
localhost linux-2.6.25.10 # make bzImage
  CHK     include/linux/version.hh
  CHK     include/linux/utsrelease.h
  CALL    scripts/checksyscalls.sh
  CHK     include/linux/compile.h
Kernel: arch/x86/boot/bzImage is ready  (#20)
localhost linux-2.6.25.10 # cp arch/x86/boot/bzImage /boot/bzImage2
cp: overwrite '/boot/bzImage2'? yes
localhost linux-2.6.25.10 # reboot
```

Result

```
#include <stdio.h>

int main(void){
        printf("hello\n");
        return 0;
}
```

위와 같은 코드를 작성한 후,

```
localhost HW # gcc my_syscall_num_test.c -o my_syscall_num_test
localhost HW # ./my_syscall_num_test
hello
localhost HW # echo 8 > /proc/sys/kernel/printk
```

마찬가지로 printk로 출력하는 내용을 볼 수 있도록 log level을 바꿔준다.

```
localhost HW # ./my_syscall_num_test
syscall[45] is called!!
syscall[33] is called!!
syscall[5] is called!!
syscall[197] is called!!
syscall[192] is called!!
syscall[6] is called!!
syscall[5] is called!!
syscall[3] is called!!
syscall[197] is called!!
syscall[192] is called!!
syscall[192] is called!!
syscall[192] is called!!
syscall[192] is called!!
syscall[6] is called!!
syscall[192] is called!!
syscall[243] is called!!
syscall[125] is called!!
syscall[125] is called!!
syscall[125] is called!!
syscall[91] is called!!
hello
syscall[119] is called!!
```

그 결과 `hello`를 출력하기 까지 필요한 system call의 number를 모두 확인할 수 있다.

Explain

3(sys_read): 파일이나 다른 입력 소스에서 데이터를 읽음
5(sys_open): 파일을 열 때 사용
6(sys_close): 파일 디스크립터를 닫음
33(sys_access): 파일이나 디렉터리에 대한 접근 권한을 확인하는 데 사용
45(sys_brk): 프로세스의 메모리 공간을 동적으로 확장 또는 축소하기 위해 사용
91(sys_munmap): 메모리 매핑을 해제하는 시스템 호출
119(sys_sigreturn): 시그널 핸들러에서 복귀
125(sys_mprotect): 메모리 보호 속성을 변경
192(sys_mmap2): 파일을 메모리에 매핑
197(sys_fstat64): 파일의 메타정보를 가져올 때 사용
243(sys_set_thread_area): 스레드의 관련 정보를 설정

HW12. What system calls are being called when you remove a file? Use "system()" function to run a Linux command as below. Explain what each system call is doing. You need to make f1 file before you run it. Also explain for each system call why that system call has been used.

HW11과 마찬가지로 파일을 만든 후 다음의 코드를 통해 지워 주었다.

```
#include <stdio.h>

int main(void){
        system("rm f1");
        return 0;
}
```

Result

```
localhost HW # ./rm_test
syscall[45] is called!!
syscall[33] is called!!
syscall[5] is called!!
syscall[197] is called!!
syscall[192] is called!!
syscall[6] is called!!
syscall[5] is called!!
syscall[3] is called!!
syscall[197] is called!!
syscall[192] is called!!
syscall[192] is called!!
syscall[192] is called!!
syscall[192] is called!!
syscall[6] is called!!
syscall[192] is called!!
syscall[243] is called!!
syscall[125] is called!!
```

...

```
syscall[192] is called!!
syscall[192] is called!!
syscall[192] is called!!
syscall[6] is called!!
syscall[192] is called!!
syscall[243] is called!!
syscall[125] is called!!
syscall[125] is called!!
syscall[125] is called!!
syscall[91] is called!!
syscall[119] is called!!
```